

Video Transcript

Video 9: Event-Driven Approach for CDC (10:01)

Change happens. Tracking it, reacting to it is one of those things that is part of life. Change is inescapable. So, when it comes to systems, it's no surprise that the same thing is true. Change data capture is a process that captures the changes made in data stores and ensures that those that care about those changes are notified. Now we created one of these by hands and we propagated that change through a number of data stores. This was the approach that we took. We made periodic queries and out of the ways that you could do it, do it yourself, DIY.

It's not a bad approach. This is architecturally the model that we had. We queried and we notified those that were interested, but I'm sure at some point you felt there were too many moving parts, and this was really a bit overwhelming. So, as it turns out, for performance reasons and others, an event-driven approach where you're reacting directly to the change as opposed to exploring for it, is much better. And as it turns out as well, most data stores keep a transaction log anyway. So, you can see there it is used to recover from failure, it is used to track changes. And the advantages are many.

One is that, it captures all changes, two is that it has low overhead as part of the operations of the data store. And a really, really great one is that it does not require database or application modifications. You don't have to touch your database, you don't have to touch the application that you're writing. And that is a great, great thing to have. Now the disadvantage is that you might have to write one of those for every type of database vendor there is. And that would be equally challenging because there are some other things that from having written one, you might recognize.

You might care about the message order that's taking place. You don't want to have robust publication and subscriptions. You want to have reliable and resilient delivery message transformations potentially. But overall, if you could do it, this would be a much better way. You have something that is tracking and flagging those events and then notifying those that care about it. Now fortunately, many of these have been written in our open source projects that you can readily use. We'll start off by looking at one in Python, but there are several others here that

we will come back to and discuss.

So, let's start off here with the one on Python, as you can see, the architecture of the application that we're going to be writing is the following. We'll start off by creating a container, and that container will have MySQL database server. Then we will connect using the workbench and add to that database, a database, a table, and some data. And then we're going to be writing a Python program that is going to be doing an implementation of this package that does CDC for us. So, let's go ahead and get started by creating the container.

The container command that we're going to use is going to be the following as you can see here. We are going to create or we're going to use the port 3306, which is the default port for MySQL, and we're going to expose it to the host, the machine that we're running in. We're going to call it some SQL. We will set the password of MyNewPass. We're going to run that detached and we're going to base it on the MySQL image. And as you can see there, that was readily done, pretty speedy since I already had the image on my machine and now let's confirm in the dashboard.

Here is the container, as you can see and note that it's up and running, it's nice and green, and that it's running on port 3306. Next, let's open up a connection to our database using the workbench. I'm going to go ahead and enter the password that I just created and here we are now at the console ready for a new script to be created. I'm going to go ahead and paste some code. As you can see here, the very first block creates a database, on line five, we are using that new database and then we are creating a table called students with three fields, email, name, and city. So, let's go ahead and run that code. And as you can see there that was successful.

Let's go ahead and reload, and you can see there that we now have an education database and a table called Students. Next, let's open up Visual Studio and let's write a program to run against the database. Now before we write our program, let's go ahead and install the drivers, install the packages that are needed. I'm going to go ahead and open up the console. The first one that I'm going to install is PyMySQL. Note that I am specifying the version.

The second one is MySQL-replication, and note that those have been satisfied on my local environment. Now let's go ahead and write the code. We're going to start off here by bringing in the Binary log StreamReader. We're then going to set our settings that we're going to use for the connection, the host, this local, the port is 3306, the user is root, and then the password is the one we set, which is MyNewPass. We're then going to write a function and note that this is the one that creates that connection and makes calls the Binary log StreamReader.

Note that, we're passing in those settings that we just defined. Then we are going to give the server an ID and then we're setting the option of blocking to true. And you can see there on line 12 that if you do that, you can wait for the next event at the end of the stream. So, in other words, it hangs and listens. And then if we get any events, we're going to go ahead and dump them to the console. So, let's go ahead and finish this up here, with a call to that function. Let's go ahead and save.

Let's go ahead and open up the console and we're going to go ahead and run here. And the program name is app.py. And as you can see there, we immediately got all of the changes that were recorded on that database from the time of creation. You can see there the events, the query events, the drop database if it exists, the create database if it doesn't exist, and finally the creation of the table, students. So, pretty fine detail of everything that is taking place. The great thing here is that we didn't have to write this ourselves.

This was written by the community and it's a package that we can leverage. Now since we're listening for the change, let's go ahead and add something to the database and see that reaction within this console. I'm going to go ahead here and make my windows smaller so I can have them side by side. So, now I have side by side, the workbench, and then I have Visual Studio and I have the console. I'm going to go ahead and make this window a little bit more roomy here by removing that side panel. And I'm going to go ahead and add some data to the student's table.

Now you can see here that I'm doing an insert into students and the values are going to be Peter and Peter Parker and then Cambridge, Massachusetts. So, I'm going to go ahead and add that. And if everything goes well, we should see the change on the right-hand side. I'm going to go ahead and do that now. And as you can see there, that change was captured. You can see there the details of the write rows event. You can see there the values of email, name, and city. And you can see there as well that if we had other parts of our program that cared about these changes, we can now notify them.

So, the great advantage here once again, is that we're not touching the application that we could be writing. We're not touching the database, we're not modifying it. We're simply bringing in a package that can do CDC by looking at the binary log record, something that the database is already doing anyway. So, this could be a great layer that we could have, that we could add that would propagate changes to any components that we had, any services that we had that would care about those changes. Now as you might imagine, this is a rich area of development. This is a

rich area of research. And there are a lot of really interesting design trade-offs that can be discussed.

And if you want to look at a great discussion on these topics, you can take a look at DBLog made by Netflix or proposed by Netflix. There's even some technical applications that you can read if you're interested. But if you want to get a synopsis and an overview of the discussion, you can navigate to the links that I have provided here for you and you can learn more about it. So, now it's your turn. We went through several steps here. We created some containers, we populated a database. We then hooked onto that database to the binary record and we were able to make updates and capture those in code that we have running on Python. So, now it's your turn. Make sure you can go through all of those steps before you go forward.

